



SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -1
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: R.Gautham Krishna

Reg. No.: 192421128

Course Code: CSA0613

Slot: A

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

Course Faculty: Dr. Mary Harin Fernandez F

Q28. Resource-Constrained Project Scheduling

1. Title

Resource-Constrained Project Scheduling Using Backtracking

2. Problem Statement

A project contains multiple tasks with different durations, resource requirements, and dependencies.

The objective is to schedule all tasks while:

- Respecting task dependencies.
- Staying within the available resource limit.
- Avoiding invalid schedules.
- Finding a feasible schedule using Backtracking.

This is a difficult optimization problem because the number of possible schedules increases rapidly as the number of tasks increases.

3. Objective

The objectives are:

- To schedule project tasks efficiently.
- To satisfy precedence constraints.
- To respect resource limitations.
- To implement Backtracking.
- To use pruning to remove invalid schedules.
- To analyze the complexity of the problem.

4. Sample Input

Consider four tasks:

Task	Duration	Resource	Dependency
------	----------	----------	------------

A	2	2	None
---	---	---	------

B	3	3	A
---	---	---	---

C	2	2	A
---	---	---	---

D	1	2	B, C
---	---	---	------

Available Resource = 4

The dependencies are:

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow D$

$C \rightarrow D$

5. Constraints

Resource Constraint

At any time:

Total Resources ≤ 4

Precedence Constraint

A task can start only after all its dependencies are completed.

For example:

A must finish before B and C.

B and C must finish before D.

6. Algorithm Used

Backtracking with Pruning

The algorithm schedules one task at a time.

If a task violates a dependency or resource constraint, the algorithm immediately rejects that choice.

This is called pruning.

7. Algorithm Steps

Step 1: Store tasks, durations, resources, and dependencies.

Step 2: Find tasks whose dependencies are completed.

Step 3: Select a valid task.

Step 4: Check the resource constraint.

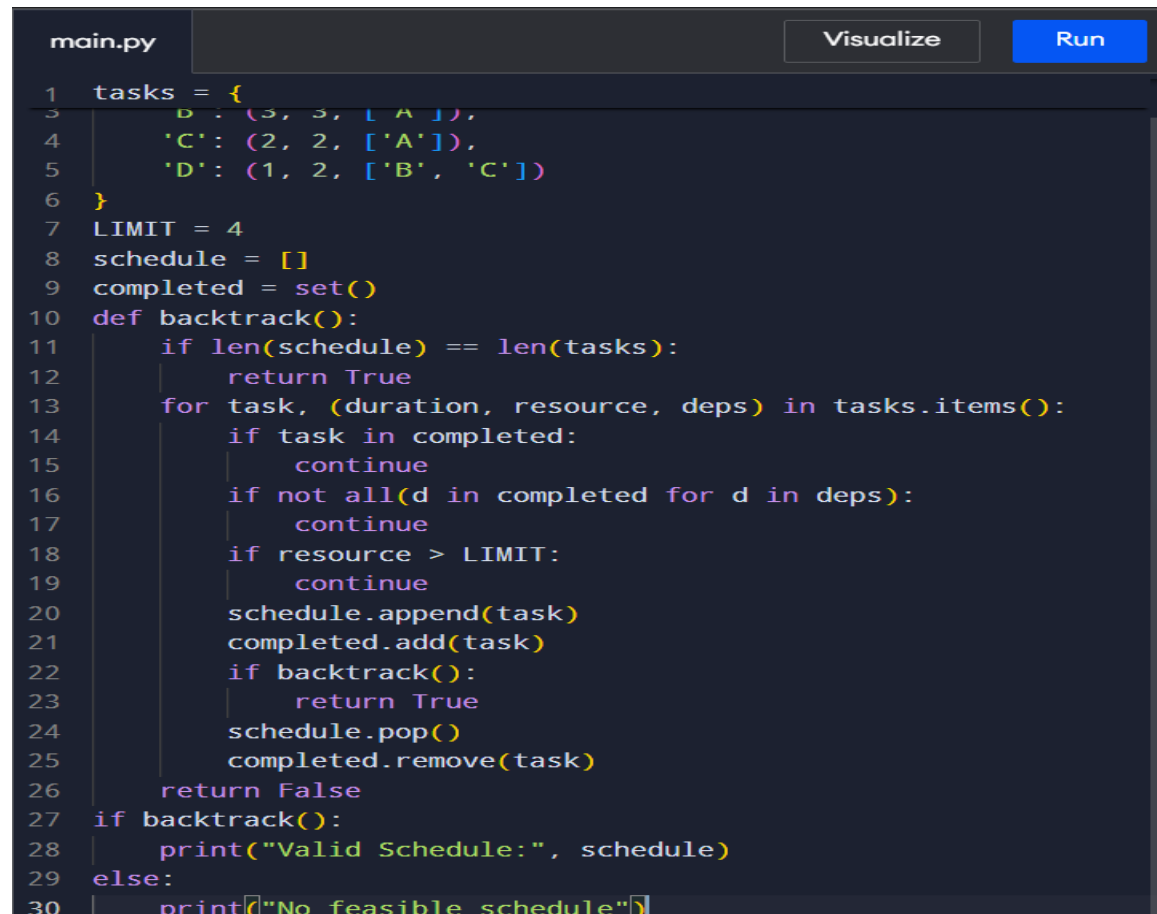
Step 5: If valid, add the task to the schedule.

Step 6: Recursively schedule the remaining tasks.

Step 7: If no valid task is available, backtrack.

Step 8: Continue until all tasks are scheduled.

8. Python Program

A screenshot of a Python IDE window titled 'main.py'. The window contains a Python script for a task scheduling algorithm. The script defines a dictionary of tasks with their durations, resources, and dependencies. It then uses a recursive backtracking function to find a valid schedule. The IDE has a dark theme and includes 'Visualize' and 'Run' buttons at the top right.

```
main.py Visualize Run
1 tasks = {
2     'B': (3, 3, ['A']),
3     'C': (2, 2, ['A']),
4     'D': (1, 2, ['B', 'C'])
5 }
6
7 LIMIT = 4
8 schedule = []
9 completed = set()
10 def backtrack():
11     if len(schedule) == len(tasks):
12         return True
13     for task, (duration, resource, deps) in tasks.items():
14         if task in completed:
15             continue
16         if not all(d in completed for d in deps):
17             continue
18         if resource > LIMIT:
19             continue
20         schedule.append(task)
21         completed.add(task)
22         if backtrack():
23             return True
24         schedule.pop()
25         completed.remove(task)
26     return False
27 if backtrack():
28     print("Valid Schedule:", schedule)
29 else:
30     print("No feasible schedule")
```

9. Output

Output

```
Valid Schedule: ['A', 'B', 'C', 'D']
```

10. Working of the Program

Initially:

Completed = {}

Task A

A has no dependency, so it is selected.

A

Task B and C

After A is completed:

B and C

can be scheduled.

Task D

D requires both B and C.

After they are completed:

D

is scheduled.

Final schedule:

$A \rightarrow B \rightarrow C \rightarrow D$

11. Pruning Technique

Pruning eliminates invalid choices before exploring them.

For example, if:

B requires A

the algorithm will not select B before A.

Similarly, if a task requires more resources than available:

Task Resource > Available Resource

that task is rejected immediately.

This reduces unnecessary searching.

12. Effect of Constraints

Constraints directly affect scheduling feasibility.

Without Constraints

Many different task orders are possible.

With Dependencies

Some task orders become invalid.

Example:

$B \rightarrow A$

is invalid because B depends on A.

With Resource Limits

Tasks requiring more resources than available cannot run simultaneously.

Therefore, constraints reduce the number of feasible schedules.

13. Backtracking Process

The algorithm follows:

Select Task

↓

Check Dependencies

↓

Check Resources

↓

Valid?

✓ ✗

Yes No

↓ ↓

Schedule Prune

↓

Next Task

If a dead end is reached:

Backtrack → Try Another Task

14. Computational Complexity

Let:

- n = Number of tasks

In the worst case, many different task orders may need to be explored.

Time Complexity – $O(n!)$

Backtracking with pruning can significantly reduce the practical search space.

Space Complexity – $O(n)$

The space is used for the schedule, completed tasks, and recursion stack.

15. NP-Hard Complexity

Resource-Constrained Project Scheduling is an NP-Hard optimization problem.

As the number of tasks increases, the number of possible schedules can grow factorially.

For example:

5 tasks → $5! = 120$ possible orders

10 tasks → $10! = 3,628,800$ possible orders

Therefore, finding the optimal schedule becomes computationally difficult for large projects.

16. Real-World Applications

This scheduling approach can be used in:

- Construction project scheduling
- Software development
- Manufacturing
- Employee scheduling
- Machine scheduling

- Maintenance planning
- Research projects

Example

In construction:

Foundation



Building Structure



Electrical Work



Finishing

Each task has dependencies and requires limited workers or machines.

17. Advantages

- Handles task dependencies.
- Respects resource limitations.
- Finds feasible schedules.
- Backtracking allows alternative schedules.
- Pruning reduces unnecessary searches.
- Suitable for small and medium scheduling problems.

18. Limitations

- Can become slow for a large number of tasks.
- Number of possible schedules grows rapidly.
- Finding the optimal solution is computationally difficult.
- Real-time scheduling becomes challenging for very large projects.

19. Result

The program successfully generates a feasible project schedule:

$A \rightarrow B \rightarrow C \rightarrow D$

All dependencies are satisfied and the resource constraint is respected.

The pruning technique prevents invalid task selections and reduces unnecessary search.

20. Conclusion

Resource-Constrained Project Scheduling can be solved using Backtracking with Pruning.

The algorithm checks resource and precedence constraints before scheduling each task.

Invalid schedules are rejected early, reducing the search space.

Since the problem is NP-Hard, the worst-case complexity grows rapidly with the number of tasks.

Time Complexity – $O(n!)$ | Space Complexity – $O(n)$